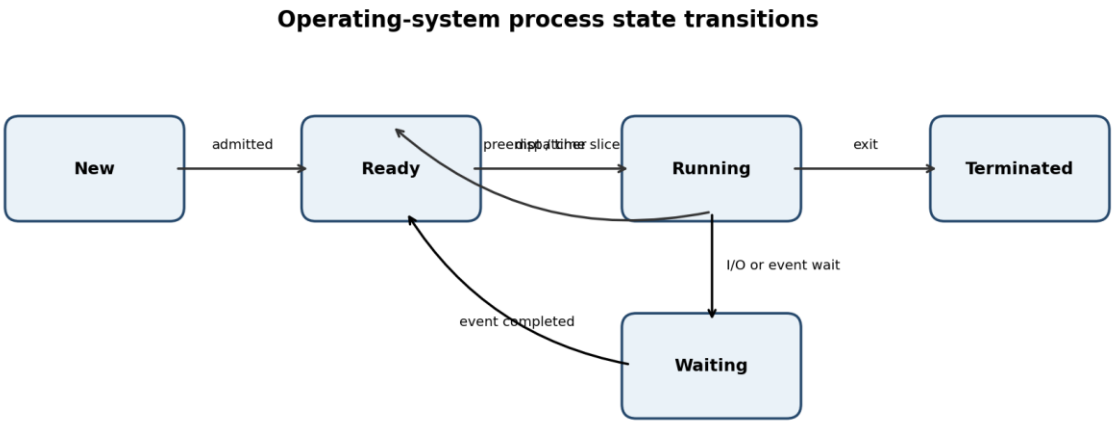
OPERATING SYSTEMS

ASSIGNMENT 2 – CO2

Process Management, Scheduling, Threads, IPC, Synchronisation and Deadlocks



Submitted in partial fulfilment of the Operating Systems course assignment

Student details	Information
Name	S. Maruthu Pandi
Register Number	192511162
Course	CSA0421 – Operating Systems
Assignment	Assignment 2 – CO2
GitHub Repository	https://github.com/Maruthupandi-maker/CSA0421-Operating-System-tree/main/CSA0421%20192511162%20Assignment%202

Q5 – SYNCHRONISATION: CRITICAL SECTIONS, SEMAPHORES AND MONITORS

5.1 Critical-section problem

A critical section is the part of a program that accesses shared data or resources. In the railway booking system, two agents may try to reserve the same seat at the same time. Without synchronisation, both may read the seat as free and both may confirm a booking. This is a race condition and can cause double booking.

A correct critical-section solution must satisfy three requirements:

- Mutual exclusion: at most one process/thread can execute the protected critical section for a particular shared resource at one time.
- Progress: if no one is in the critical section and some participants want to enter, the decision about who enters next cannot be postponed indefinitely by processes that are not interested in entering.
- Bounded waiting: after a process requests entry, there must be a finite bound on how many times others can enter before it gets its turn.

5.2 Peterson's solution for two processes

Peterson's algorithm uses two shared intent flags and a turn variable. flag[i] indicates that process i wants to enter. turn gives the other process priority when both want to enter simultaneously.

```
// Shared variables, initially false and 0:
bool flag[2] = {false, false};
int turn = 0;

// Code executed by process i; j = 1 - i
flag[i] = true;
turn = j;
while (flag[j] && turn == j) {
    /* busy wait */
}
/* critical section */
flag[i] = false;
/* remainder section */
```

For two processes under the algorithm's memory model assumptions, only one can pass the while condition at a time (mutual exclusion). If one process is not interested, the other can enter (progress). The turn variable prevents one process from entering repeatedly while the other is waiting (bounded waiting).

On modern multiprocessor systems, compiler and CPU memory reordering can make the simple textbook code unsafe if ordinary loads and stores do not provide the required ordering. A correct implementation needs suitable atomic variables and memory-ordering fences. In production C programs, a pthread mutex or well-tested atomic synchronisation primitive is normally preferred.

5.3 Binary and counting semaphores

A semaphore is an integer synchronisation object accessed through atomic operations wait (P/down) and signal (V/up). wait decrements the semaphore when a resource is available or blocks the caller; signal increments it and may wake a waiting thread.

Semaphore type	Value meaning	Railway example
Binary semaphore	Usually 0 or 1; can protect a single critical section when used consistently.	A binary semaphore or mutex protects the seat-status update.

Counting semaphore	Non-negative count of identical available resources.	A semaphore initialised to the number of available booking agents or service slots can limit concurrent use.

For a seat, the protected operation is: acquire the seat lock; check whether the seat is free; if free, mark it booked and record the customer; otherwise report unavailable; release the lock. The check-and-update must be one atomic critical section.

5.4 Bounded-buffer producer-consumer problem

A producer inserts items into a finite buffer and a consumer removes them. Three synchronisation objects are commonly used: empty counts free slots (initial value SIZE), full counts filled slots (initial value 0), and mutex protects the buffer indices and array.

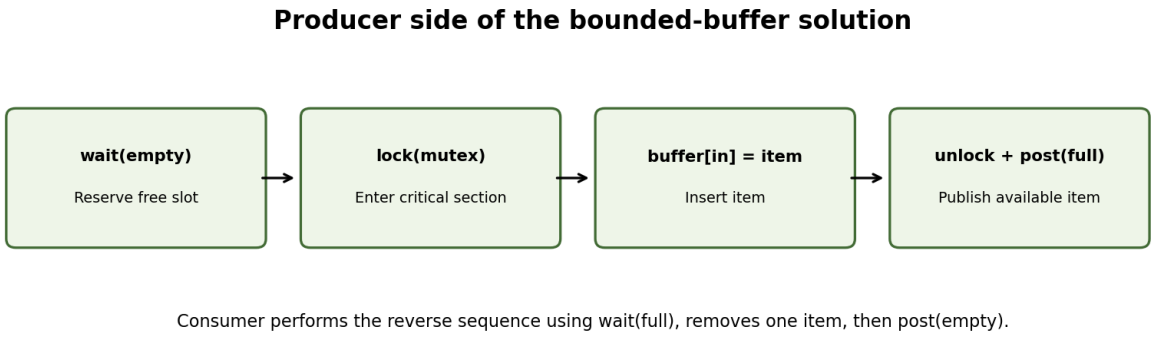


Figure 5. Producer sequence: reserve a slot, lock, insert, unlock and publish the item.

5.5 Completed semaphore and mutex program

The following program completes both TODO sections from the assignment. The producer waits for an empty slot, enters the critical section, stores an item and signals full. The consumer waits for a full slot, removes an item and updates the sum while holding the mutex.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define SIZE 5
#define ITEMS 20
int buffer[SIZE], in = 0, out = 0;
sem_t empty, full;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
long consumed_sum = 0;

void *producer(void *arg) {
   (void)arg;
   for (int item = 1; item <= ITEMS; ++item) {
      sem_wait(&empty);
      pthread_mutex_lock(&lock);
      buffer[in] = item;
      in = (in + 1) % SIZE;
      pthread_mutex_unlock(&lock);
      sem_post(&full);
   }
   return NULL;
}

void *consumer(void *arg) {
   (void)arg;
   for (int k = 0; k < ITEMS; ++k) {
      sem_wait(&full);
      pthread_mutex_lock(&lock);
      int item = buffer[out];
      out = (out + 1) % SIZE;
      consumed_sum += item;
```

```
        pthread_mutex_unlock(&lock);
        sem_post(&empty);
    }
    return NULL;
}

int main(void) {
    if (sem_init(&empty, 0, SIZE) != 0 || sem_init(&full, 0, 0) != 0) {
        perror("sem_init"); return 1;
    }
    pthread_t p, c;
    if (pthread_create(&p, NULL, producer, NULL) != 0 ||
        pthread_create(&c, NULL, consumer, NULL) != 0) {
        fprintf(stderr, "pthread_create failed\n"); return 1;
    }
    pthread_join(p, NULL); pthread_join(c, NULL);
    printf("items consumed: %d, sum = %ld (expected %d)\n",
           ITEMS, consumed_sum, ITEMS*(ITEMS+1)/2);
    sem_destroy(&empty); sem_destroy(&full);
    pthread_mutex_destroy(&lock);
    return 0;
}
```

Compile and run:

```
gcc -std=c11 -Wall -Wextra -pthread q5_buffer.c -o q5_buffer
./q5_buffer
```

Expected output: items consumed: 20, sum = 210 (expected 210). Run it repeatedly with a shell loop:

```
for i in $(seq 1 100); do ./q5_buffer; done
```

The sum should be 210 on every run. In this one-producer/one-consumer version the mutex protects buffer indices and the sum. With multiple producers and consumers, every shared

update still needs protection. The total number of items and termination signals must also be coordinated so consumers do not wait forever after all producers finish.

5.6 What happens if the mutex is removed?

The empty/full semaphores still count slots and items, but they do not by themselves protect the compound updates to in, out and the shared buffer. With multiple producers or consumers, two threads can read the same index before either updates it, causing them to overwrite or consume the same slot. The consumed_sum variable can also lose updates if several consumers update it concurrently. The program may appear to work in some runs and fail in others; correctness cannot be inferred from a single successful output.

5.7 Readers-writers problem

The readers-writers problem models a shared database in which many readers may read simultaneously, but a writer must have exclusive access. A common solution maintains a reader count under a mutex. The first reader locks the database against writers; the last reader unlocks it. Writers lock the database before changing data. A reader-preference solution can starve writers, so a fair read-write lock or writer-preference policy may be required for a booking database.

5.8 Dining-philosophers problem

In the dining-philosophers problem, each philosopher needs two adjacent forks. If every philosopher picks up one fork and waits for the other, a circular wait occurs. Solutions include imposing a global order on fork acquisition (always take the lower-numbered fork first), allowing at most $N-1$ philosophers to compete simultaneously, or using a monitor to grant both forks atomically. These methods break the possibility of circular wait or prevent all participants from holding one resource while waiting for another.

5.9 Monitors and condition variables

A monitor is a high-level synchronisation construct that combines shared data and operations that access it. Only one thread executes inside the monitor at a time. Condition variables allow a thread to wait until a condition becomes true. In a railway booking system, a monitor can encapsulate the seat array and make the “check whether the seat is available and reserve it” operation atomic.

```
monitor SeatBooking {
    bool occupied[SEAT_COUNT];
    condition seatAvailable;

    bool tryBook(int seat, Customer c) {
        if (occupied[seat]) return false;
        occupied[seat] = true;
        owner[seat] = c;
        return true;
    }

    void cancel(int seat) {
        occupied[seat] = false;
        signal(seatAvailable);
    }
}
```

The pseudocode expresses the monitor concept; exact syntax differs between languages. A condition wait must always be used in a loop that rechecks the predicate, because a wake-up does not itself guarantee that the seat is still available.

5.10 Lock granularity: global, per train or per coach

Locking choice	Concurrency	Risk / overhead	Recommendation
One global lock	Only one booking operation across the whole system can modify state at a time.	Simple and safe but creates a bottleneck; unrelated trains block one another.	Avoid for a large booking service except for small global metadata.
One lock per train	Bookings on different trains can proceed concurrently; seats on the same train are serialised.	Good balance; contention may remain on a very popular train.	Recommended starting design.

One lock per coach	Bookings in different coaches can proceed concurrently.	Highest concurrency but requires careful protection of train-wide rules, coach allocation and cancellation consistency.	Use only after measurement and when operations are truly independent.

Throughput must be measured using the same number of booking threads, train/coach distribution and number of operations. Record successful bookings per second and ensure that the final seat count has no duplicates. No numerical throughput is supplied here because it must be collected on the student's machine.

Lock strategy	**Measured operations/s**	**Duplicate-seat test**	**Notes**
Global lock	_____	Pass / Fail	_____
Per train	_____	Pass / Fail	_____
Per coach	_____	Pass / Fail	_____

5.11 Atomic operations and spin locks alternative

A C11 atomic_fetch_add operation performs a single atomic read-modify-write, making it suitable for a simple counter. Its work per update is O(1), but contention can still cause cache-line traffic. A spin lock repeatedly checks whether a lock is free; it avoids sleeping and waking for very short critical sections, but wastes CPU time while waiting. A mutex or semaphore can block a thread, which is more suitable for longer critical sections.

Method	**Best use**	**Concern**
Atomic counter	A single independent counter update.	Cannot protect a multi-step invariant involving several variables by itself.
Spin lock	Very short critical section on a system where spinning is acceptable.	Wastes CPU cycles under contention; can hurt performance if held for long.

Mutex / semaphore	Longer critical sections or waits where blocking is appropriate.	May incur scheduling and wake-up overhead, but avoids continuous spinning.

Expected measurement plan: measure average update time with 1, 4 and 16 threads for each method; also record process CPU time. If thread count increases and spin time rises sharply, use a blocking synchronisation primitive. For a long critical section, mutex/semaphore is normally preferable to a spin lock.